

Lab experience 4: Bloodless pulse oximeter

Objectives

- Introduce and apply software-controlled multiplexing in data acquisition
- Implement a feedback control system via LabVIEW using analog input & output
- Create a two-LED photoplethysmograph (device that uses light to measure volume changes)
- (Optional) Upgrade your photoplethysmograph so that it auto-adjusts its illumination levels to accommodate a range of tissue thicknesses.

Catching up

The starting point for this lab exercise is the dual-LED driver circuit and VI that you created for Lab 3 (**Lab3_squarewave_yourname.vi**). It is also assumed that you completed the oscilloscope in Lab 3, but that VI is not necessary here.

Exercises (In - Lab)

Demonstrate each VI for the instructor, and record data or answer the questions the document write up handout linked on canvas.

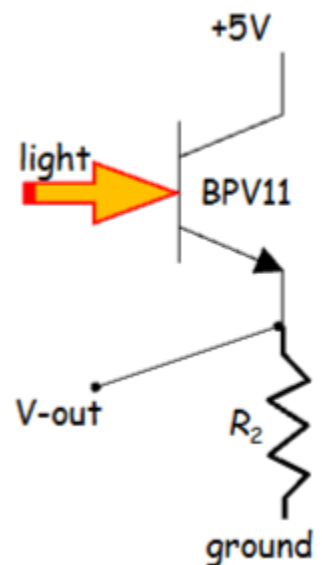
0. Recreate your dual-LED circuit from Lab 3
1. Create a photodetector circuit for acquisition by the DAQ hardware and LabVIEW.

Your photodetector circuit will use a silicon phototransistor and be added to your dual-LED circuit. Create a voltage divider in which the phototransistor and R_2 are in series from the supply voltage to ground. A supply voltage in the 6-9 V range works well (**make sure the current is limited to 40-50 mA**), and the output should be measured across R_2 . You may wish to attach the phototransistor at the end of an extension cable so it can be aimed at the LEDs.

2. Add data acquisition to your dual-LED VI

Start with your VI that produces the alternating light output from your two LEDs. Within the While loop, add a DAQ Assistant that measures the voltage output from your photodetector circuit. **You may find it helpful to name your DAQ Assistants, so you can keep track of what is being input to/output from your VI.** Commercial pulse oximeters sample each LED at 100 Hz, so a brightness value should be captured 200 times per second (100×2 LEDs). However, LabVIEW is not particularly fast over USB, so let's collect measurements half that fast. That is, set the wait time of your while loop to 10 ms (or higher).

Use a case structure that determines which LED is turning on and which is turning off, and a shift register to save/display the data points for both LEDs (see Figure 1).



For the boolean case of the case structure, the acquisition sequence is the following: transition, then signal capture, then the opposite transition, then the complementary signal capture (i.e. 0, 1, 0, 1). You created this pattern using a formula node to create your alternating square wave. Replicate this logic using low-level Labview functions (hint: Quotient & Remainder = modulo function) **[GREEN WIRES ARE BOOLEAN WIRES]**. Include the Mean function found in Mathematics > Probability and Statistics > Mean with the result of one piece of information per LED per loop. You will then need to build a 2D array of dimensions 1x2 that passes both channels of data simultaneously into a chart (you did this in Lab 3, when plotting the voltages that you are supplying to the transistors & LEDs).

In order to reduce the effect of high-frequency noise that might cause error in a given measurement, collect multiple points from your DAQ, then average them and store the average in an output array. With a phototransistor, it is possible to sample at the necessary rate; 1 kHz should work well.

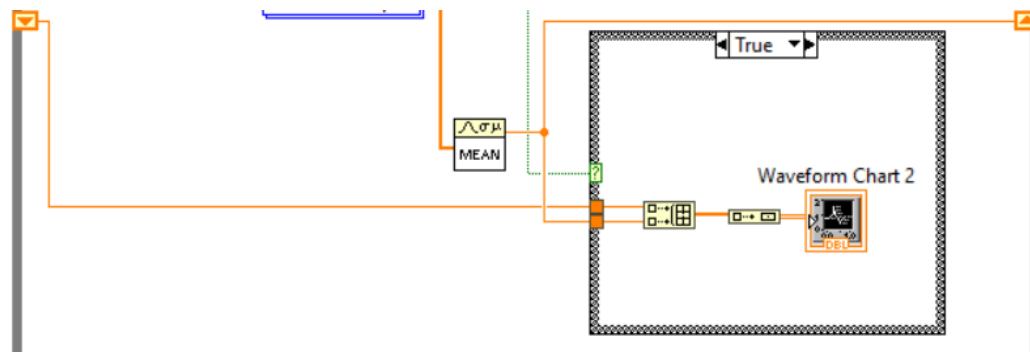


Figure 1. Plotting two LEDs as separate curves on a chart. False case is empty and not shown.

3. Measure heart rate with IR

After confirming your device is working properly, ask the instructors for: (1) a fingertip pulse oximeter, (2) an IR LED, and (3) an IR phototransistor.

3. Replace one of the color LEDs in your circuit with the IR LED.
4. Use the BPW96 phototransistor (clear lens, peak sensitivity: 850 nm)
5. Most likely the offset between the two LED signals are too big to allow for a close-up view on the waveform chart. In which case you may want to add another waveform chart to just look at the signal from the IR LED.
 - Adjust your input chart such that it is displaying a phototransistor signal of 10 seconds long (Try changing the x-axis to accommodate 10 x fs samples). Count the number of pulses in this 10-second window. Multiply it by 6 to get the beats per minute.
 - It may help to clear the waveform chart after each run. To do this, create a History property node outside the loop, change it to write, and create a constant

6. Place your finger between the IR LED and the phototransistor in a dark environment to observe pulsation activities on your VI. Based on experience, we find it helpful to place the phototransistor near the crescent-shape whitish area of the nail, and place the IR LED on the other side of the finger.

Open 468_Lab04_2026_LabViewPulseOx_Document_WriteUp to begin your write up for this lab.

Deliverables:

- Get instructor check-off on your device
- Upload your documented VI (and any sub-VIs you used).
- Upload the write-up as a PDF document.

Optional exercise

Add a brightness auto-adjust feature (described here) OR the “ratio of ratios” analysis used in a pulse oximeter (described in lecture)

Background: Feedback control

Feedback control is discussed at length in BIOEN 336 and 337. Briefly, control is the process of “steering” a system (sailboat, automobile, autoclave, LED, blood plasma) to maintain some variable (direction, speed, temperature, brightness, drug concentration) by applying some input (rudder force, engine power, heat, current, drug injection). Feedback control consists of measuring the relationship between the desired value and the actual value of the variable, then taking the appropriate action (or inaction).

An example of the simplest feedback controller is a thermostat that causes a heater to turn on when the temperature is below some setpoint and turn off when it exceeds the setpoint. A more complicated, but still common, controller produces an input (e.g., motor speed) that is proportional to the difference between the desired and actual conditions; this is a proportional controller. Another level of sophistication includes integral and derivative terms to determine the controller output (system input), improving the controller performance; this is known as a PID controller, meaning proportional-integral-derivative.

The LED controller for this week is not quite in either category, because we don’t want the LED brightness to produce a constant light intensity after passing through the finger – such a system would cancel out the effect of the pulsations! Instead, we want the LED brightness to produce a detector signal that fills as much of the ADC range as practical without ever exceeding its upper limit. The controller should take in the detector signal and a two thresholds (high and low); if the detected light intensity is above the upper threshold, the system reduces the amount of light produced by the LEDs, while when it is below a (possibly different) threshold the light intensity is increased. The increase and decrease should be by some significant chunk (e.g. $\times 0.8$ or $\times 1.25$) so the brightness is not continually being adjusted.

Feedback programming activity

The goal of the auto-adjust feature is to use as much of the ADC voltage range as is practical without exceeding the maximum ADC input voltage. The lower voltage limit is not a concern because the output from our voltage divider cannot drop below 0 V. This handout suggests that you adjust the brightness by some factor according to whether the detector output is above or below some threshold, but you could use preset brightness levels if you prefer.

- a. Select upper and lower thresholds for the photodetector output; these will be incorporated as controls or constants in your VI.
- b. Select an initial digital-to-analog convertor output (i.e. the voltage that is passed to the base of each 2N3904 transistor). Initiate your VI using this value.

- c. During each iteration, after acquiring and averaging the signal from each LED, the VI should compare the maximum value from the photodetector circuit to the upper and lower thresholds. If it exceeds the upper threshold, multiply the DAC output voltage by a factor less than 1 (e.g. 0.8); if it is below the lower threshold, multiply by a factor greater than 1 (e.g. 1.25). Use this new output voltage for the succeeding iteration of the While loop.
- d. Repeat this process of comparison and adjustment until the maximum photodetector voltage is between the two thresholds. Note that in both cases we are testing the maximum voltage; the minimum voltage is not problematic.
- e. The VI should plot the most recent data that were in range (or the initial values, if the VI has just been turned on and no in-range data have been collected yet).
- f. When the voltage is beyond one of the thresholds, the VI should display an appropriate indicator (e.g. a Boolean LED on the front panel).

The VI may be written to analyze each LED individually or to compare and adjust both LEDs as a pair.

If you complete some or all of the Optional Exercise, include one paragraph in your write-up, stating what you accomplished and the results you achieved. Good work!